

Self-Attention in Transformer Architecture: A Deep Dive

Why Self-Attention? Motivation and Core Intuition

Self-attention replaced recurrence and convolution for sequence modeling because it simultaneously solves three bottlenecks: parallelization, long-range dependency capture, and inductive bias flexibility.

Parallelization. RNNs process tokens sequentially—each step depends on the previous hidden state—so training cannot be parallelized across time. CNNs allow parallel convolution but require stacking many layers to expand the receptive field, making very long contexts expensive. Self-attention computes all token interactions in a single matrix multiplication, enabling full sequence parallelism on GPUs/TPUs.

Long-range dependencies. In an RNN, gradients must flow through many time steps, leading to vanishing/exploding gradients. CNNs need depth proportional to sequence length to connect distant tokens. Self-attention gives every token a direct path to every other token in one layer, so the gradient path length is constant ($O(1)$) regardless of distance.

Inductive bias. RNNs assume a strong sequential bias (order matters, recent tokens matter more). CNNs assume locality and translation invariance. Self-attention is permutation-equivariant: it treats the sequence as a set and learns relational structure from data, making it adaptable to tasks where the relevant structure isn't known a priori.

Key-Value-Query as Retrieval

Think of each token as generating a **query** (what I'm looking for), a **key** (what I offer), and a **value** (the content I carry). Attention scores are dot-products between queries and keys, then softmaxed to form a probability distribution. The output for a token is the weighted sum of values—exactly a soft retrieval from the sequence where the query decides relevance.

Dynamic, Context-Dependent Routing

Because queries and keys are computed from the current representations, the routing changes at every layer and for every input. A token can attend to different tokens depending on context (e.g., "bank" attends to "river" vs. "money"). This dynamic routing is what lets the model disambiguate polysemy and model complex syntactic/semantic relations without hand-crafted features.

Concrete Example

Consider the sequence: "The animal didn't cross the street because it was too tired".

- The pronoun "it" generates a query that strongly matches the key of "animal" (semantic compatibility) and weakly matches "street".
- Attention weights might be: animal: 0.7, street: 0.1, others: 0.2.
- The output for "it" becomes a weighted blend of the value vectors, dominated by "animal", giving the model the information needed to resolve the coreference.

This token-to-token interaction, computed in parallel for all positions, is the core primitive that makes transformers both expressive and hardware-efficient.

Mathematical Formulation: From Scaled Dot-Product to Multi-Head

Query, Key, Value Projections

Self-attention begins by projecting the input sequence $X \in \mathbb{R}^{n \times d_{\text{model}}}$ into three matrices: queries Q , keys K , and values V . Each projection is a learned linear layer:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$. Typically $d_k = d_v = d_{\text{model}}/h$ for h heads, so each head operates on a lower-dimensional subspace. This split allows the model to attend to different representation aspects in parallel.

Scaled Dot-Product Attention

For a single head, attention weights are computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

The dot product $QK^\top$ measures similarity between queries and keys. Without scaling, large d_k causes the dot products to grow in magnitude, pushing the softmax into regions with vanishing gradients. Dividing by $\sqrt{d_k}$ keeps the variance of the dot products roughly constant regardless of d_k , stabilizing training. The softmax produces a probability distribution over keys for each query, and the weighted sum of values yields the output.

Multi-Head Attention

Multi-head attention runs h independent attention heads in parallel:

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i), \quad i = 1 \dots h$$

Each head has its own projection matrices W_Q^i, W_K^i, W_V^i . The head outputs are concatenated and projected back to d_{model} :

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

with $W_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$. This design lets the model jointly attend to information from different representation subspaces at different positions.

Residual Connection and Layer Normalization

Each sub-layer (attention or feed-forward) is wrapped in a residual connection followed by layer normalization:

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

The residual path preserves gradient flow and enables deep stacking. Layer normalization stabilizes activations across the feature dimension, which is critical because attention outputs can have varying scales depending

on the input sequence. In the original Transformer, normalization is applied *after* the residual addition (post-norm). Some variants (e.g., Pre-LN) place normalization before the sub-layer for improved training stability at scale.

Minimal Implementation: Self-Attention from Scratch

Below is a compact, dependency-light PyTorch module that implements scaled dot-product attention and multi-head attention with explicit shape annotations. The code uses `einops.rearrange` for readable head splitting/merging and includes a causal mask for autoregressive decoding.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from einops import rearrange

class ScaledDotProductAttention(nn.Module):
    def forward(self, q, k, v, mask=None):
        # q, k, v: (batch, seq_len, d_model)
        d_k = q.size(-1)
        scores = torch.matmul(q, k.transpose(-2, -1)) / (d_k ** 0.5) # (batch, seq_len, seq_len)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float('-inf'))
        attn = F.softmax(scores, dim=-1)
        return torch.matmul(attn, v) # (batch, seq_len, d_model)

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model: int, n_heads: int, dropout: float = 0.1):
        super().__init__()
        assert d_model % n_heads == 0
        self.d_model = d_model
        self.n_heads = n_heads
        self.d_head = d_model // n_heads

        self.q_proj = nn.Linear(d_model, d_model, bias=False)
        self.k_proj = nn.Linear(d_model, d_model, bias=False)
        self.v_proj = nn.Linear(d_model, d_model, bias=False)
        self.out_proj = nn.Linear(d_model, d_model, bias=False)
        self.attn = ScaledDotProductAttention()
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        # x: (batch, seq_len, d_model)
        batch, seq_len, _ = x.shape

        q = self.q_proj(x)
        k = self.k_proj(x)
        v = self.v_proj(x)

        # Split heads: (batch, seq_len, n_heads, d_head) -> (batch, n_heads, seq_len, d_head)
        q = rearrange(q, 'b s (h d) -> b h s d', h=self.n_heads)
```

```

k = rearrange(k, 'b s (h d) -> b h s d', h=self.n_heads)
v = rearrange(v, 'b s (h d) -> b h s d', h=self.n_heads)

# Apply causal mask if provided (shape: batch, 1, seq_len, seq_len)
if mask is not None:
    mask = mask.unsqueeze(1) # expand for heads

out = self.attn(q, k, v, mask) # (batch, n_heads, seq_len, d_head)
out = rearrange(out, 'b h s d -> b s (h d)') # merge heads
return self.out_proj(self.dropout(out))

# Quick shape verification
if __name__ == "__main__":
    batch, seq_len, d_model, n_heads = 2, 10, 512, 8
    x = torch.randn(batch, seq_len, d_model)
    # Causal mask: lower triangular (1 = attend, 0 = masked)
    causal_mask = torch.tril(torch.ones(seq_len, seq_len)).bool()
    mha = MultiHeadAttention(d_model, n_heads)
    out = mha(x, causal_mask)
    assert out.shape == (batch, seq_len, d_model), f"Expected {(batch, seq_len, d_model)}, got {out.shape}"
    print("Shape test passed:", out.shape)

```

Key points

- **Shape discipline:** Every tensor carries `(batch, seq_len, d_model)` until head splitting, where we explicitly reshape to `(batch, n_heads, seq_len, d_head)`.
- **Einops rearrange:** Replaces error-prone `view/transpose` chains with declarative patterns.
- **Causal masking:** The lower-triangular mask is broadcast across heads via `unsqueeze(1)`; `masked_fill` with `-inf` ensures zero attention to future positions.
- **Test tensor:** The `__main__` block confirms the output shape matches the input `(batch, seq_len, d_model)`.

Edge cases

- If `d_model` isn't divisible by `n_heads`, the `assert` fails early.
- Passing a mask with incorrect broadcast dimensions (e.g., missing head dimension) will raise a runtime shape error.
- For very long sequences, the `seq_len × seq_len` attention matrix may OOM; consider flash attention or chunked implementations in production.

Computational Complexity and Memory Optimization

FLOPs and Memory: Attention vs. Feed-Forward

Self-attention dominates compute and memory at long sequence lengths. For a sequence length N and model dimension d (with h heads of dimension $d_h = d/h$):

Component	FLOPs (approx.)	Activation Memory
Attention (QK ^T , softmax, AV)	$4N^2d$	$O(N^2)$ for $S = QK^T$

Component	FLOPs (approx.)	Activation Memory
Feed-Forward (two matmuls)	$8Nd^2$	$O(Nd)$

When $N \gg d$, the N^2 term overwhelms the Nd^2 term. For example, at $N = 8192$, $d = 4096$, attention FLOPs are ~ 1.1 TFLOPs vs. ~ 1.1 TFLOPs for FFN—comparable, but attention requires ~ 256 MiB for the $N \times N$ score matrix (fp16) while FFN needs only ~ 64 MiB. At $N = 32768$, attention memory exceeds 4 GiB, becoming the primary bottleneck.

Flash Attention: Kernel Fusion and Tiling

Flash Attention eliminates the materialized $N \times N$ matrix by fusing the entire attention computation into a single GPU kernel. It tiles Q, K, V into blocks that fit in SRAM (shared memory):

1. Load a block of K, V into SRAM.
2. Iterate over Q blocks: compute $S_{ij} = Q_i K_j^T$, apply softmax online via the **online softmax** trick (maintaining running max and sum), and accumulate $O_i = \sum_j \text{softmax}(S_{ij}) V_j$.
3. Write O_i to HBM once.

This reduces HBM reads/writes from $O(N^2)$ to $O(Nd)$ and avoids the N^2 memory allocation. The kernel also fuses dropout and causal masking.

```
# Pseudocode: online softmax for a single query block
def flash_attn_block(Q_block, K_blocks, V_blocks):
    O_block = zeros_like(Q_block)
    m_i = -inf; l_i = 0
    for K_j, V_j in zip(K_blocks, V_blocks):
        S_ij = Q_block @ K_j.T          # (Bq, Bk)
        m_ij = max(m_i, S_ij.max(dim=-1))
        P_ij = exp(S_ij - m_ij[:, None])
        l_ij = P_ij.sum(dim=-1)
        # Rescale previous output and accumulate
        O_block = O_block * exp(m_i - m_ij[:, None]) + P_ij @ V_j
        l_i = l_i * exp(m_i - m_ij) + l_ij
        m_i = m_ij
    return O_block / l_i[:, None]
```

Key-Value Caching for Autoregressive Generation

During decoding, each layer caches K and V for all previous tokens. For a model with L layers, h heads, head dimension d_h , and batch size B , the cache size at step t is:

$$\text{Memory} = 2 \times L \times B \times t \times h \times d_h \times \text{bytes}$$

With fp16 (2 bytes), $L = 32$, $h = 32$, $d_h = 128$, $B = 1$, the cache grows by **256 KiB per token**. At 32k tokens, this reaches ~ 8 GiB. Techniques like **quantized KV cache** (int8/fp8) or **multi-query/grouped-query attention** (sharing K, V across heads) reduce this footprint by 2–8 $\times$.

Sparse Attention Patterns

Sparse patterns approximate full attention with $O(N)$ or $O(N\sqrt{N})$ complexity:

Pattern	Complexity	Description	Trade-off
Local (sliding window)	$O(Nw)$	Each token attends to w neighbors	Loses long-range dependencies; good for local syntax
Strided	$O(N \cdot N/s)$	Attend every s -th token	Captures periodic structure; may miss fine-grained relations
Block-sparse	$O(N \cdot b)$	Dense blocks of size $b \times b$	Flexible; requires learned/fixed sparsity pattern; hardware support varies

Longformer combines local + global tokens; BigBird adds random blocks. All sacrifice exactness for scalability—choose based on task dependency range and hardware sparsity support.

Edge Cases, Failure Modes, and Numerical Stability

Attention Collapse and Entropy Regularization

Attention collapse occurs when the softmax output becomes nearly uniform, causing the model to lose token discrimination. This often appears early in training or when query/key projections are poorly initialized. A practical fix is to add an entropy regularization term to the loss:

```
# entropy regularization (add to loss)
attn_weights = F.softmax(scores, dim=-1)
entropy = -(attn_weights * attn_weights.clamp(min=1e-9).log()).sum(-1).mean()
loss += lambda_ent * entropy
```

Increasing `lambda_ent` (e.g., 1e-3) pushes the distribution toward lower entropy, encouraging sharper attention. Monitor average entropy per layer; values near `log(seq_len)` indicate collapse.

Softmax Overflow: Log-Sum-Exp and FP32 Accumulation

Large logits produce `NaN/Inf` in `exp`. The standard remedy is the log-sum-exp trick computed in FP32 even when the model runs in BF16/FP16:

```
def stable_softmax(x, dim=-1):
    x_fp32 = x.float()
    max_val = x_fp32.max(dim=dim, keepdim=True).values
    exp_x = (x_fp32 - max_val).exp()
    return (exp_x / exp_x.sum(dim=dim, keepdim=True)).to(x.dtype)
```

Always accumulate the reduction in FP32. Most frameworks (PyTorch, JAX) now do this internally for `nn.functional.softmax`, but custom kernels must implement it explicitly.

Position Encoding Extrapolation Limits

- **Absolute sinusoidal**: Fails beyond trained sequence length; frequencies are fixed.
- **Relative biases (T5, ALiBi)**: Generalize better but still degrade at extreme lengths.
- **RoPE**: Rotary embeddings extrapolate more gracefully, yet performance drops when test length $\gg$ train length. Mitigate by scaling the base frequency (θ) at inference (e.g., $\theta * (\text{train_len} / \text{test_len})$) or using YaRN/LongRoPE fine-tuning.

Gradient Explosion in Deep Stacks

Deep attention stacks amplify gradient variance. Use **depth-scaled initialization**:

- Scale query/key/output projections by $1 / \sqrt{2 * \text{num_layers}}$ (or $1 / \sqrt{\text{num_layers}}$) for pre-LN).
- Apply **LayerScale** (learnable per-layer scalar initialized to $1e-3$) on the residual branch.
- Clip global gradient norm at 1.0.

These measures keep the gradient norm stable across 100+ layers without sacrificing convergence speed.

Attention Variants and Architectural Extensions

Cross-attention vs. self-attention. Self-attention computes relationships within a single sequence (encoder or decoder). Cross-attention, used in encoder-decoder architectures, lets the decoder attend to the encoder's output, enabling sequence-to-sequence tasks. In decoder-only models, cross-attention is absent; the decoder relies solely on causal self-attention.

Grouped-query attention (GQA) and multi-query attention (MQA). Both reduce the number of key/value heads to accelerate inference:

- **MQA** uses a single key/value head shared across all query heads. It maximizes memory bandwidth savings but can degrade quality on complex tasks.
- **GQA** groups query heads (e.g., 8 query heads share 2 key/value heads), offering a tunable trade-off between MQA's speed and multi-head attention's expressiveness. GQA is now standard in many large language models (e.g., Llama 2 70B) for efficient KV-cache usage.

Linear attention and kernel approximations. Standard attention scales quadratically with sequence length. Linear attention rewrites the softmax as a kernel feature map $\phi(\cdot)$ so that attention becomes $(QK^T)V = Q(K^TV)$, reducing complexity to $O(N)$. Common kernels include:

- **Elu+1** or **ReLU** feature maps for positive-definite kernels.
- **Random Fourier features** for shift-invariant kernels.
- **Performer** and **Linear Transformer** implementations use these to achieve sub-quadratic scaling, though they may struggle with tasks requiring sharp attention distributions.

Attention sinks and sliding window attention. For long-context models, two complementary techniques manage memory and compute:

- **Attention sinks** (e.g., in StreamingLLM) reserve a few initial tokens as "sinks" that accumulate global attention, preventing performance collapse when the KV cache is evicted.
- **Sliding window attention** restricts each token to attend only to a fixed window of recent tokens (e.g., 4k tokens), yielding linear scaling. Combined with sinks, this enables stable, constant-memory inference on arbitrarily long sequences.

Debugging and Observability: Visualizing Attention Patterns

Extract and visualize attention heatmaps

Most frameworks expose attention weights via forward hooks or model outputs. In PyTorch, register a hook on `nn.MultiheadAttention` or the transformer block to capture `attn_output_weights` (shape `[batch, heads, seq_len, seq_len]`). For Hugging Face models, set `output_attentions=True`. Plot heatmaps per layer and head using `matplotlib` or `seaborn` for a single input sequence; this reveals which tokens each head attends to.

```
# Minimal hook to capture attention weights
attn_weights = {}
def hook(module, input, output):
    attn_weights['last'] = output[1] # (batch, heads, seq, seq)
model.transformer.layers[3].self_attn.register_forward_hook(hook)
_ = model(input_ids)
```

Identify head specialization via probing

Run targeted probes on a validation set: feed sentences with known syntactic dependencies (subject-verb, adjective-noun) and compute the average attention weight between dependent tokens. Heads that consistently assign high weight to such pairs are syntactic specialists. Positional heads show strong diagonal or fixed-offset patterns; semantic heads attend to content words across long distances. Cluster heads by their probe scores to assign functional labels.

Detect degenerate patterns with entropy and distance metrics

Compute per-head attention entropy: $H = -\sum(p * \log p)$ over the query dimension. Low entropy (< 0.5) indicates collapsed attention (e.g., all mass on `[CLS]` or a single token). High entropy (> 2.0) suggests under-trained or noisy heads. Track average attention distance: $\sum(\text{weight} * |i-j|)$. A sudden drop to near zero signals positional collapse; a rise beyond sequence length may indicate padding leakage.

Log statistics during training

Add a callback that records per-layer, per-head mean entropy, max weight, and mean distance every N steps. Alert when any head's entropy falls below a threshold for three consecutive checkpoints. Correlate these metrics with loss spikes to catch attention collapse early, before it degrades downstream task performance.

[IMAGE GENERATION FAILED] Figure 1: Multi-head self-attention computation flow. Input X is projected to Q, K, V; each head computes scaled dot-product attention independently; outputs are concatenated and projected back to `d_model`.

Alt: Multi-head self-attention data flow: input projections to Q/K/V, scaled dot-product per head, concatenation and output projection

Prompt: Technical diagram showing multi-head self-attention flow: input tensor X (batch, seq_len, d_model) splits into three linear projections W_Q, W_K, W_V producing Q, K, V. Each splits into h heads (batch, n_heads, seq_len, d_head). Each head computes $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$.

Head outputs concatenated and projected via W_O . Clean boxes and arrows, labeled tensor shapes, minimal color.

Error: kroki: HTTP Error 400: Bad Request | gemini: 429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: <https://ai.google>.'}}

[IMAGE GENERATION FAILED] Figure 2: Flash Attention kernel tiling. Q, K, V are split into blocks fitting in SRAM. For each Q block, iterate over K/V blocks: compute S_{ij} , update running max m_i and sum l_i (online softmax), accumulate output O_i . Avoids materializing $N \times N$ matrix in HBM.

Alt: Flash Attention block-wise computation: Q, K, V tiles loaded to SRAM, online softmax with running max/sum, output accumulated in registers

Prompt: Technical diagram of Flash Attention tiling: Large Q, K, V matrices divided into blocks. Arrows show Q_{block_i} loaded to SRAM, then loop over K_{block_j} , V_{block_j} loaded to SRAM. Compute $S_{ij} = Q_i K_j^T$ in registers. Online softmax: running max m_i , running sum l_i , rescale accumulator O_i . Final O_i written to HBM. Show SRAM vs HBM boundary. Minimal colors, clear labels.

Error: kroki: HTTP Error 400: Bad Request | gemini: 429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: <https://ai.google>.'}}

[IMAGE GENERATION FAILED] Figure 3: Sparse attention patterns. (a) Local/sliding window: each token attends to w neighbors. (b) Strided: attend every s -th token. (c) Block-sparse: dense $b \times b$ blocks. (d) Longformer: local + global tokens. Patterns shown as $N \times N$ attention masks (dark = attend).

Alt: Sparse attention patterns: local sliding window, strided, block-sparse, and global+local (Longformer) visualized as attention matrices

Prompt: Four small heatmap-style diagrams side by side showing sparse attention patterns as $N \times N$ matrices (dark pixels = attention allowed). (a) Local/sliding window: diagonal band. (b) Strided: vertical stripes every s columns. (c) Block-sparse: scattered dense squares. (d) Longformer: diagonal band + full rows/cols for global tokens. Axes labeled 'query position' and 'key position'. Clean, minimal, technical style.

Error: kroki: HTTP Error 400: Bad Request | gemini: 429 RESOURCE_EXHAUSTED. {'error': {'code': 429, 'message': 'You exceeded your current quota, please check your plan and billing details. For more information on this error, head to: <https://ai.google>.'}}